CS & IT ENGINEERING



Algorithms

Divide & Conquer



Lecture No.- 01

By- Aditya sir

Recap of Previous Lecture



- Topic
- Topic

Sorting

No class
12, 13

Class
10, 11 → 8 AM

Topics to be Covered



- Topic
- Topic
- Topic

Divide & Conquer



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.

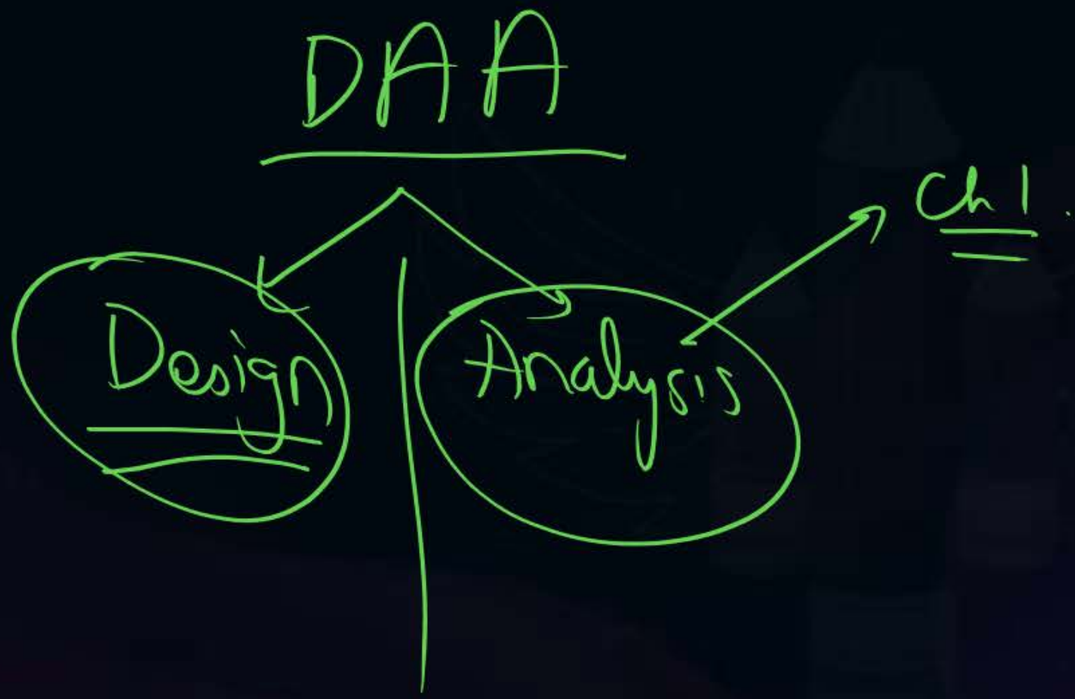


Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Design Strategies

1. Divide & Conquer (D & q)
2. greedy Algo
3. Dynamic programming → (DP)

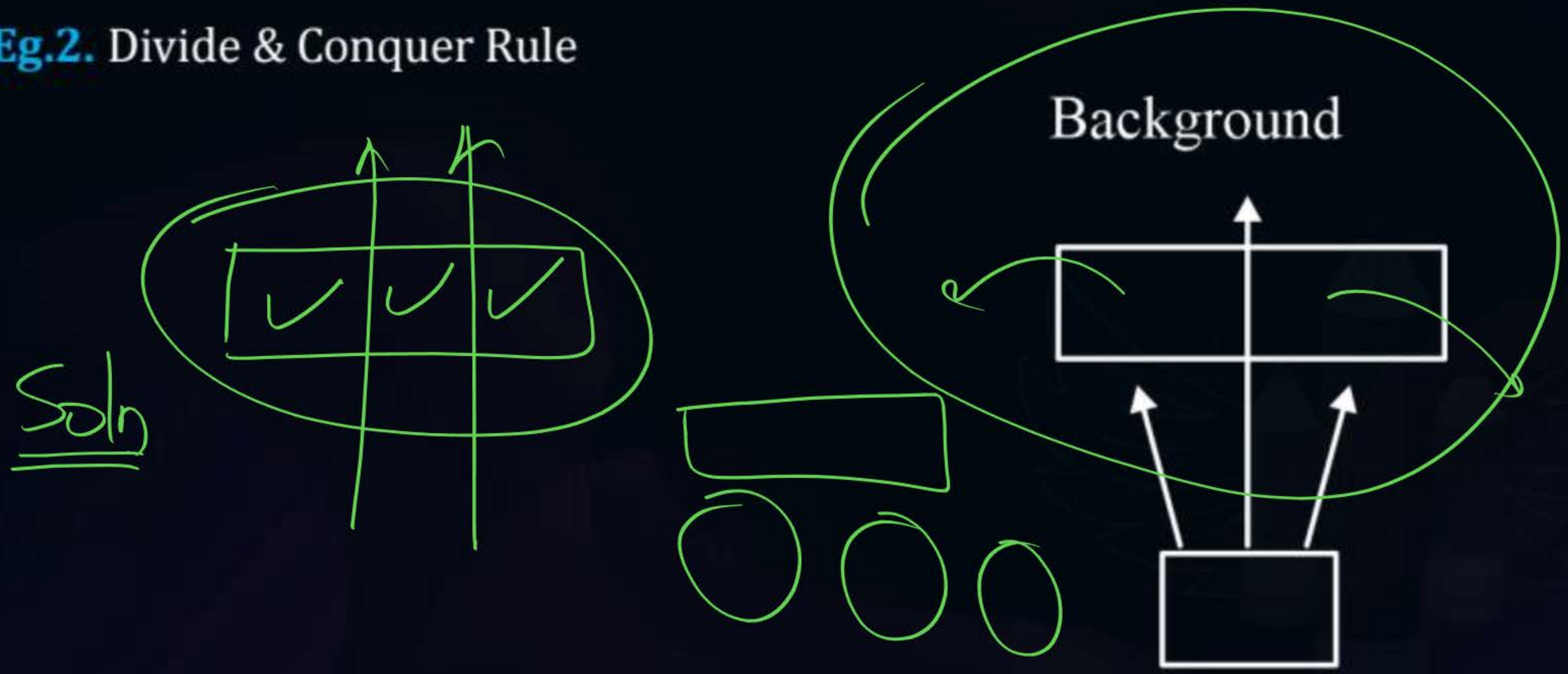




Topic : Design Strategies

Britisher Rule

Eg.2. Divide & Conquer Rule





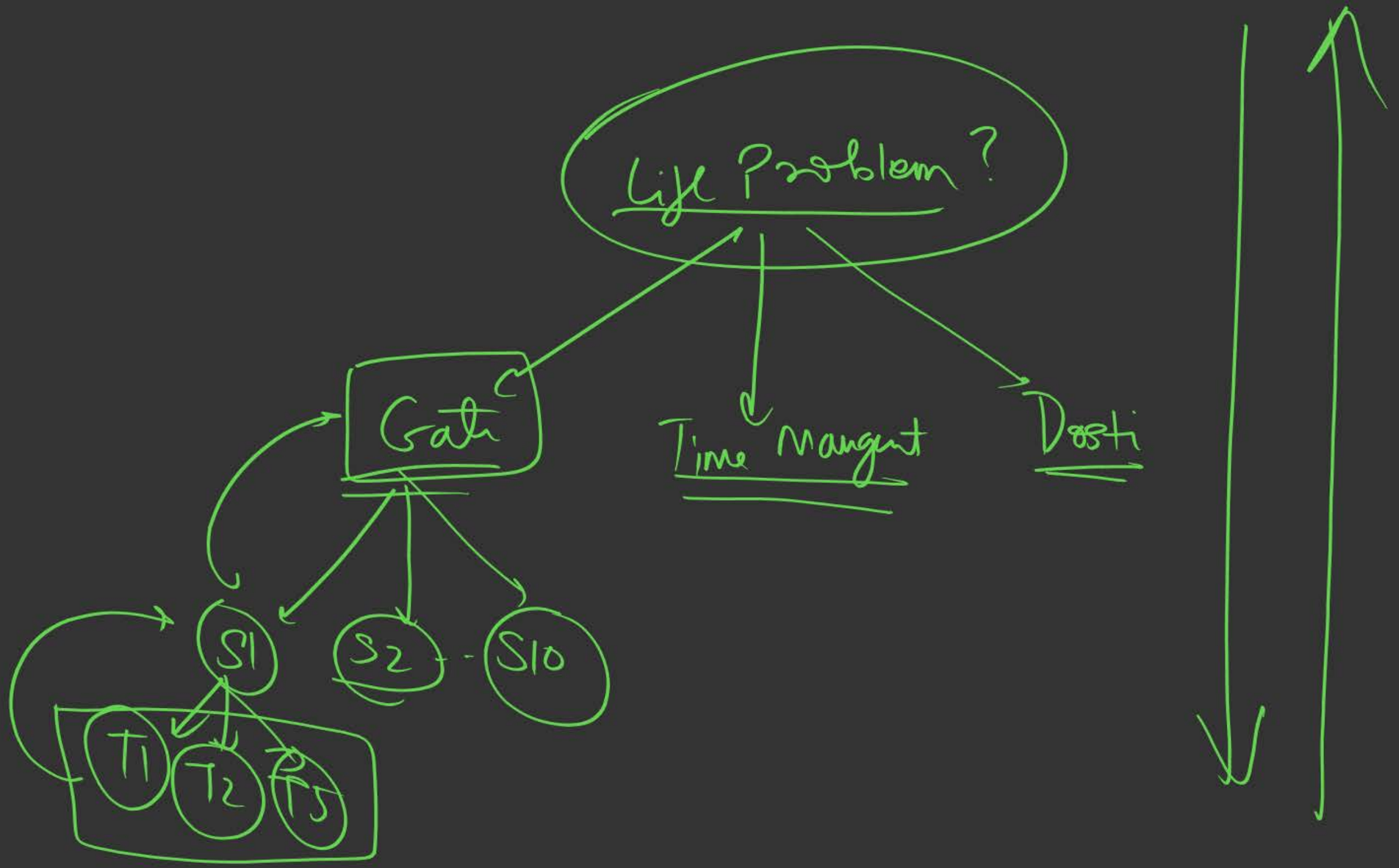
Topic : Design Strategies

Divide & Conquer

→ When the problem becomes large/complex, then divide the problem into subproblems, into further subproblems, until the subproblem becomes small.

→ Solve the smaller problem combine their results if required to get the solution of original problem.

→ In general a problem is said to be small, if it can be solved in one/two basic operations.


Life Problem?
Gath
Time Mangmt
Dosti
S1
S2
S10
T1
T2
T3



Topic : Design Strategies

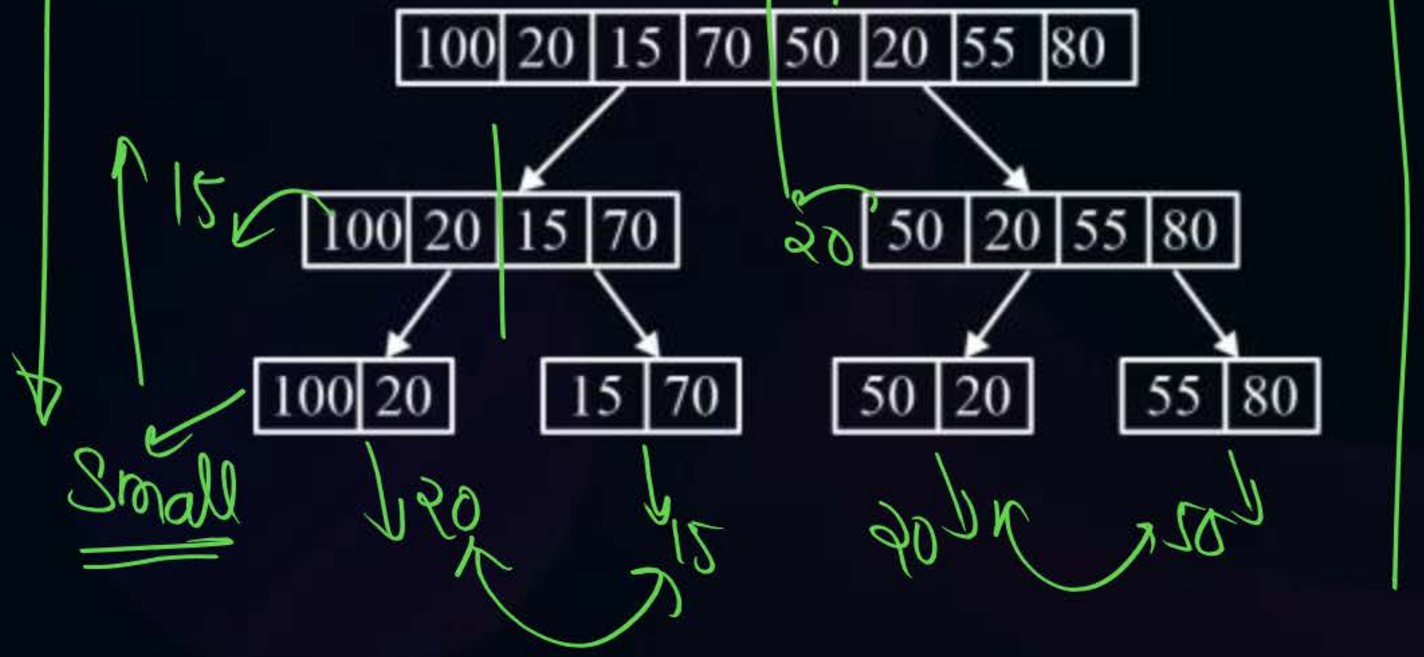
#Q. Given an array of elen, find the min element.

Idea:-

Ans: 15

min(15, 20) = 15

n = 8



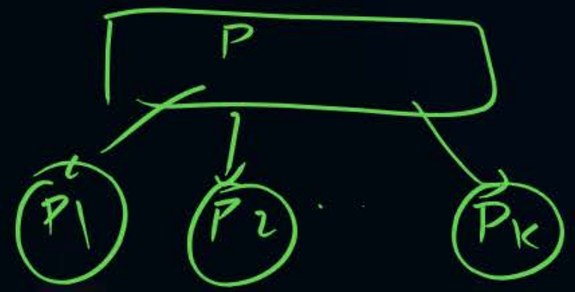




Topic : Design Strategies

Framework / Idea

```
1.  Algorithm DAndC(P)
2.  {
3.      if Small(P) then return S(P);
4.      else
5.      {
6.          divide P into smaller instances P1, P2, ..., Pk, k ≥ 1;
7.          Apply DAndC to each of these subproblems;
8.          return Combine(DAndC(P1),DAndC(P2), ... , DAndC(Pk));
9.      }
10. }
```

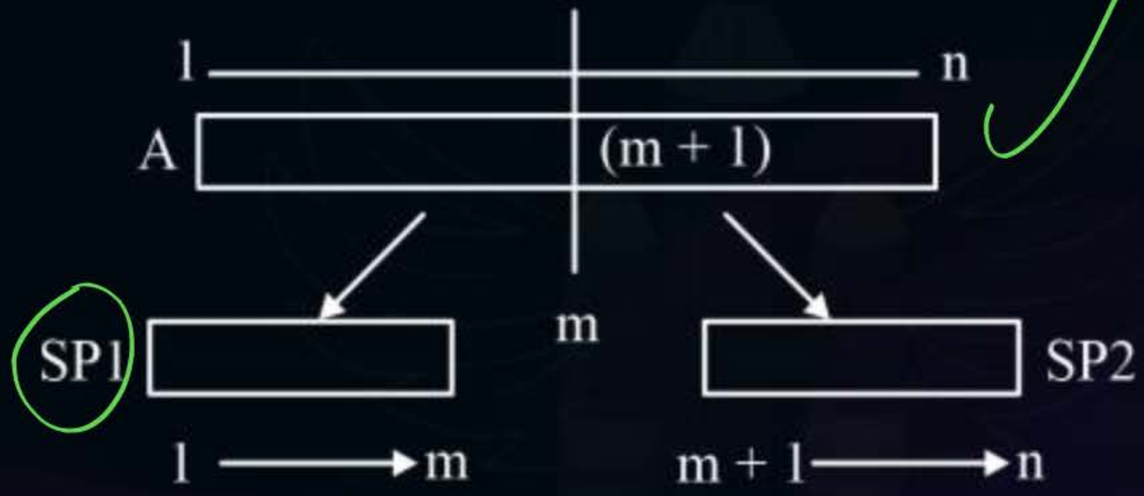




Topic : Design Strategies

for 2 subproblems

```
Algorithm DandC (A, 1, n)
{
        if (SMALL (1, n))
        return (S(A, 1, n));
        else
        {
        m ← DIVIDE (1, n)
        S1 ← DandC (A, 1, m)
        S2 ← DandC (A, m + 1, n)
        Combine (S1, S2);
}
```





Topic : Design Strategies

Time complexity for divide & conquer problem:-

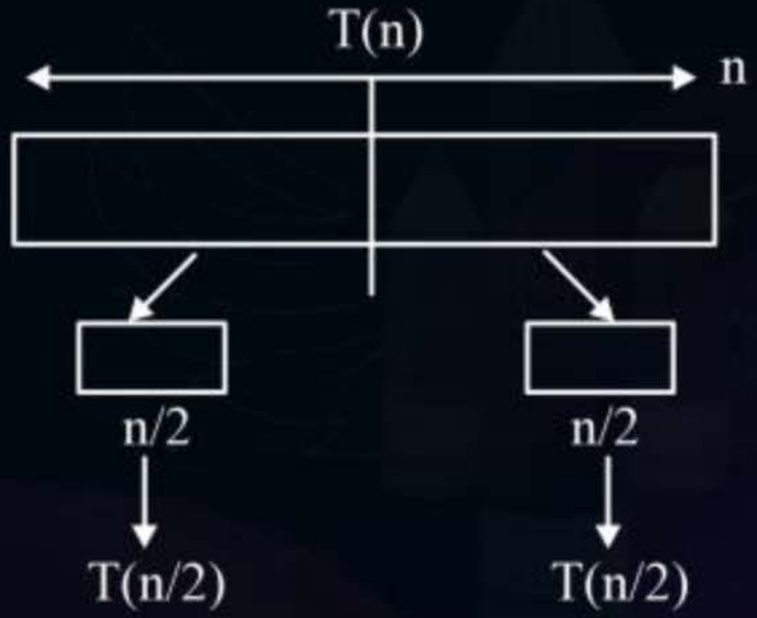
Let T(n) → original problem P.

T(n) = g(n), if n is small → usually Const

T(n) = 2*T(n/2) + F(n), if n is large

↓

Time Reqd for D&C





Topic : Design Strategies

Computing time of DAndC is described by the recurrence relation

$$T(n)\begin{cases} g(n) & \text{n samll otherwise} \\ T(n_1)+T(n_2)+\ldots+T(n_k)+f(n), & \text{otherwise} \end{cases}$$

In general

DAC



Topic : Design Strategies

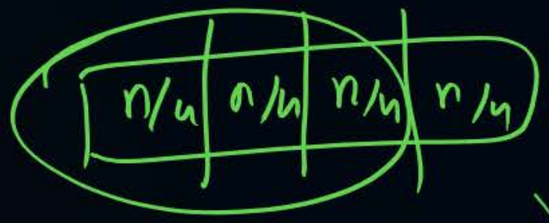
Recurrence Relations in Divide & Conquer Problem

(1) Symmetric

(2) Asymmetric → Type-1, Type - 2



Topic : Design Strategies



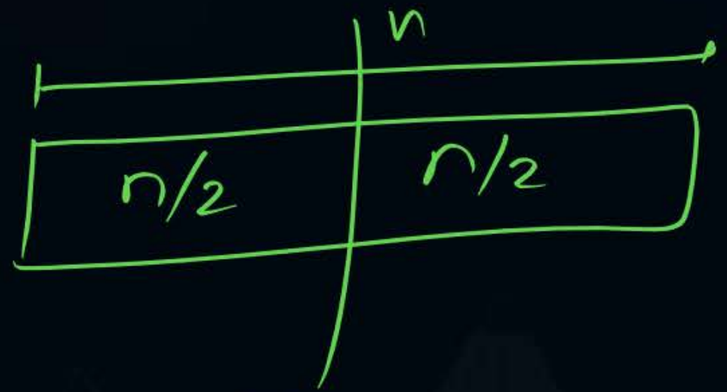
(1) Symmetric D & C

General form: $T(n) = a * T(n/b) + F(n)$

Eg.2. $T(n) = 2T(n/2) + n$

$T(n) = 3T(n/4) + n^2$

$T(n) = 2T(n/3) + n^3$



a → no. of subproblems.

n/b → size of each subproblem.

Master's Thm



Topic : Design Strategies

(2) Asymmetric D & C Type: -1

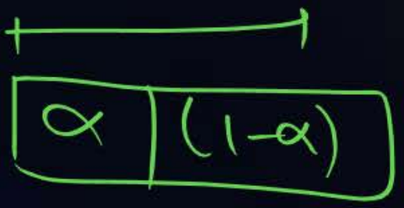
Eg. $T(n) = T(n/4) + T(3n/4) + F(n)$

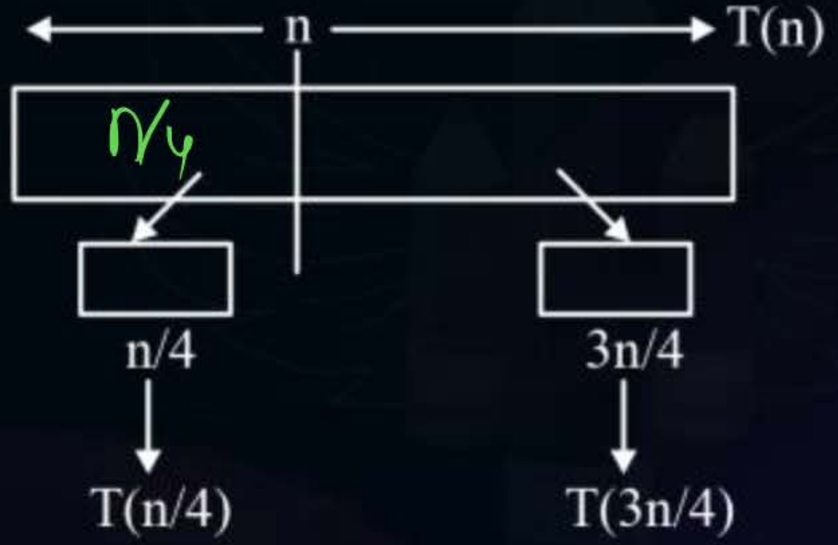
Eg. $T(n) = T(n/3) + T(2n/3) + g(n)$

$n - n/4 = 3n/4$

General form

$T(n) = T(\alpha n) + T((1-\alpha)n) + g(n)$





cost of up

Recursion Tree Approach:



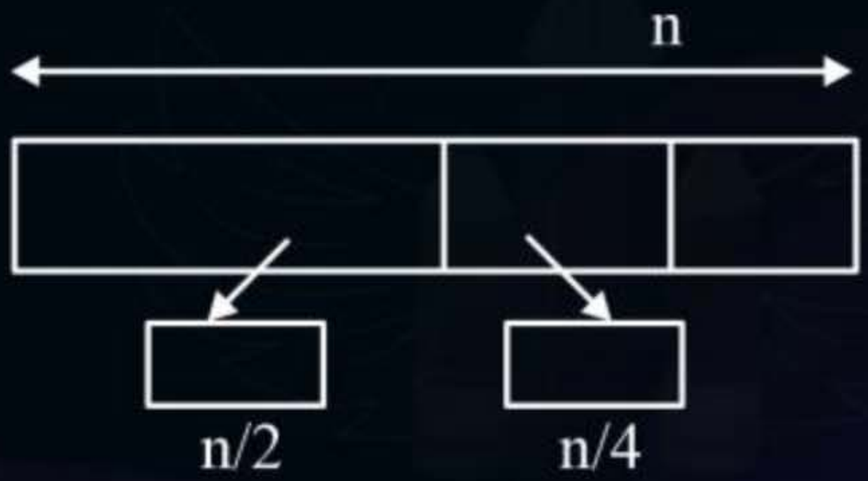
Topic : Design Strategies

(2) Asymmetric D & C Type: -2

Eg. $T(n) = T(n/2) + T(n/4)$

General

$T(n) = T(\alpha n) + T(\beta n) + T(\delta n) \ldots g(n)$



Recursion Tree Approach:



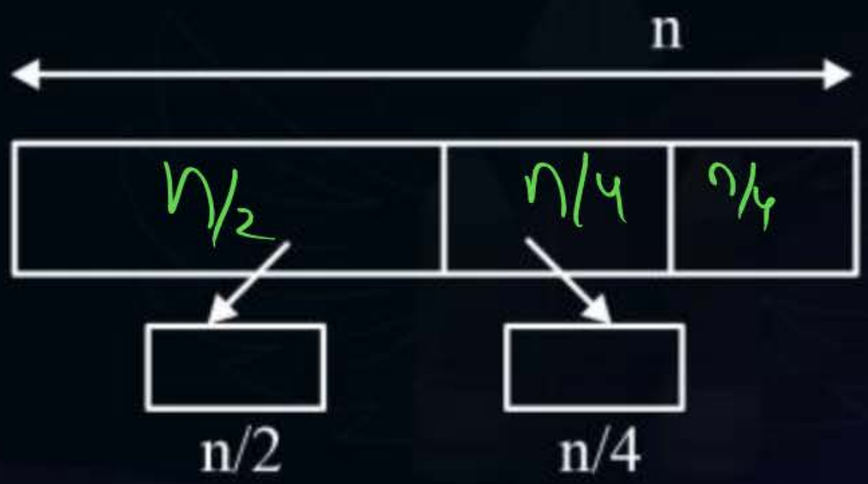
Topic : Design Strategies

(2) Asymmetric D & C Type: -2 → Rare

Eg. $T(n) = T(n/2) + T(n/4)$

General

$T(n) = T(\alpha n) + T(\beta n) + T(\delta n) \ldots + g(n)$



Recursion Tree Approach:



Topic : Design Strategies

Problem → Divide & Conquer:- (Bonus)

Divide → Mandatory

Conquer → optional

Eg. Binary Search

→ Divide

→ But no need to combine





Topic : Design Strategies

(1) Min – Max Problem

Max - Min



Topic : Design Strategies

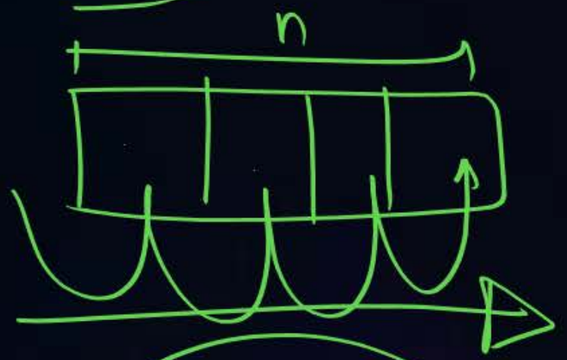
(1) Min – Max Problem:

Find min & max element of the array simultaneously

2p

min elem

n



min_elem =

2p

max elem

In one go



Topic : Design Strategies

Appr I :- Non – D & C Based

Eg.

7	15	–1	–6	17	1	8	59	29	24
1	2	3	4	5	6	7	8	9	10

Algo AJ – Min Max (A, n)

```
min = max = A[1]
for(i = 2; i ≤ n; i++)
if(A[i] < min)
{
    min = A[i]
}
if(A[i] > max)
{
    max = A[i]
}
```

(n − 1) times

No. of Comparisons :

↓

element - wise

Total no. of elemt-wise Comp in this ?

= 2 * (n − 1) = 2n − 2.



Topic : Design Strategies

Non-Dn C

```
Algo    AI – min-max (A, n)
        min = max = A[1]
        for(i = 2; i < n; i++)
        {
            if(A[i] < min)
            min = A[i]
            else if (A[i] > max)
            max = A[i]
        }
```

3

3

for(i = 2; i < n; i++) → (n-1) times

B.C Comp → if (true) → (n-1) Comp

W.C Comp → if (false)

if → (n-1) + (n-1) → else-if

= 2 * (n-1)



Topic : Design Strategies

Time complexity analysis of Aglo2

(1) Best case: input in Decr order

Eg.

10	7	5	2

→ (n-1) Comp

(2) Worst case: input in Incr order

Eg.

2	5	7	10

→ 2*(n-1) Comp

Topic : Design Strategies

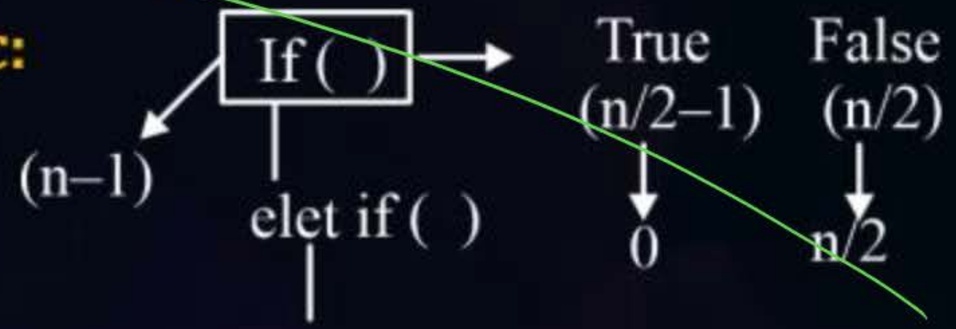
Bonus: Avg Case: V.V.V. Imp

On an avg, if condition fails 1/2 of the times.

$$(n-1)+\frac{n}{2}$$

$$= n-1+\frac{n}{2} = \frac{3n-2}{2} = \boxed{\frac{3n}{2}-1}$$ **

Logic:



if → (n-1)

else if → (n/2)

n = 10 elems

$\frac{3\times10}{2} - 1 = 3\times5-1$

$= 14$



Topic : Design Strategies

Summary:

V. Imp

Non-DnC based Min-Max Algo

Imp

	if	else if	Total Comparision
Best case	((n–1)	0) ⇒	(n–1)
Wrost case	(n–1)	(n–1) ⇒	2*(n–1)
Avg case	(n–1)	n/2 ⇒	$\frac{3n-1}{2}$

Ω(n) — O(n) } Θ(n)

Imp

T.C



Topic : Time Complexity

#Q.
```
for (i = 2; i ≤ n; i = i²)
    for (j = 1 ; j ≤ n; j ++)
        for (k = 1; k ≤ n; k = k + j)
            x = y + z;
```

→ log(logn)

nested dependent

→ O(nlogn)

overall : O(nlogn * log(logn))

A. O(log(logn))

B. O(nlogn)

C. O(log(nlogn)) → 30%

D. O(log(logn)*nlogn) → 44%

$i = 2$; $i <= n$; $i = i^2$

k^{th} ⟶ last iter

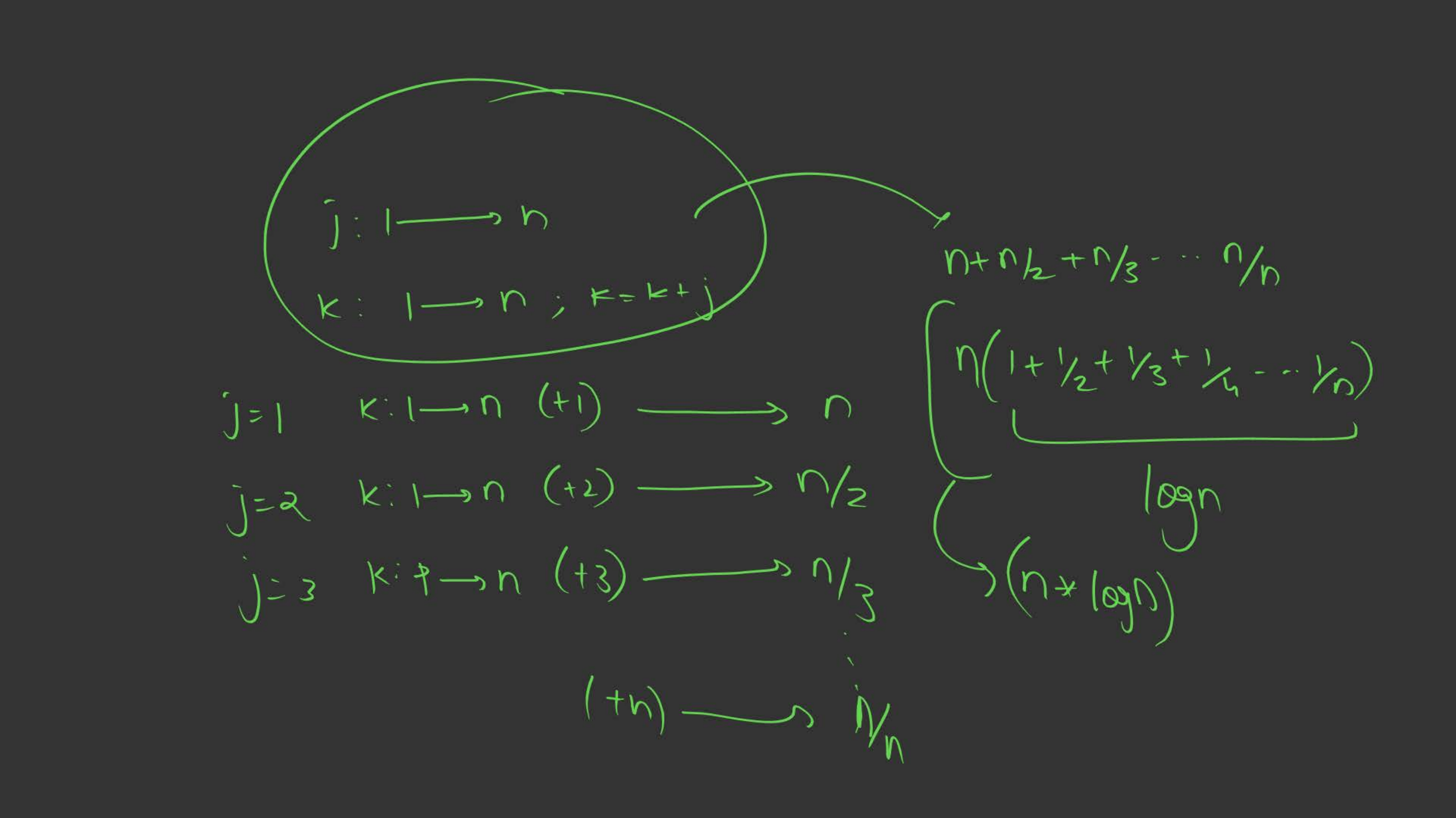
$2 \rightarrow 2^2 \rightarrow 4^2 \rightarrow 16^2 \ldots \ldots \; (2^{2^k})$

4 16

For last iter $2^{2^k} = n$

$2^k \times \log_2 2 = \log_2 n$

$$k = \log_2(\log_2 n)$$


j: 1 ⟶ n
k: 1 ⟶ n ; k = k + j
n + n/2 + n/3 ⋯ n/n
n(1 + 1/2 + 1/3 + 1/4 ⋯ 1/n)
log n
(n * log n)
j = 1 k: 1 ⟶ n (+1) ⟶ n
j = 2 k: 1 ⟶ n (+2) ⟶ n/2
j = 3 k: 1 ⟶ n (+3) ⟶ n/3
(+n) ⟶ n/n



Topic : Time Complexity

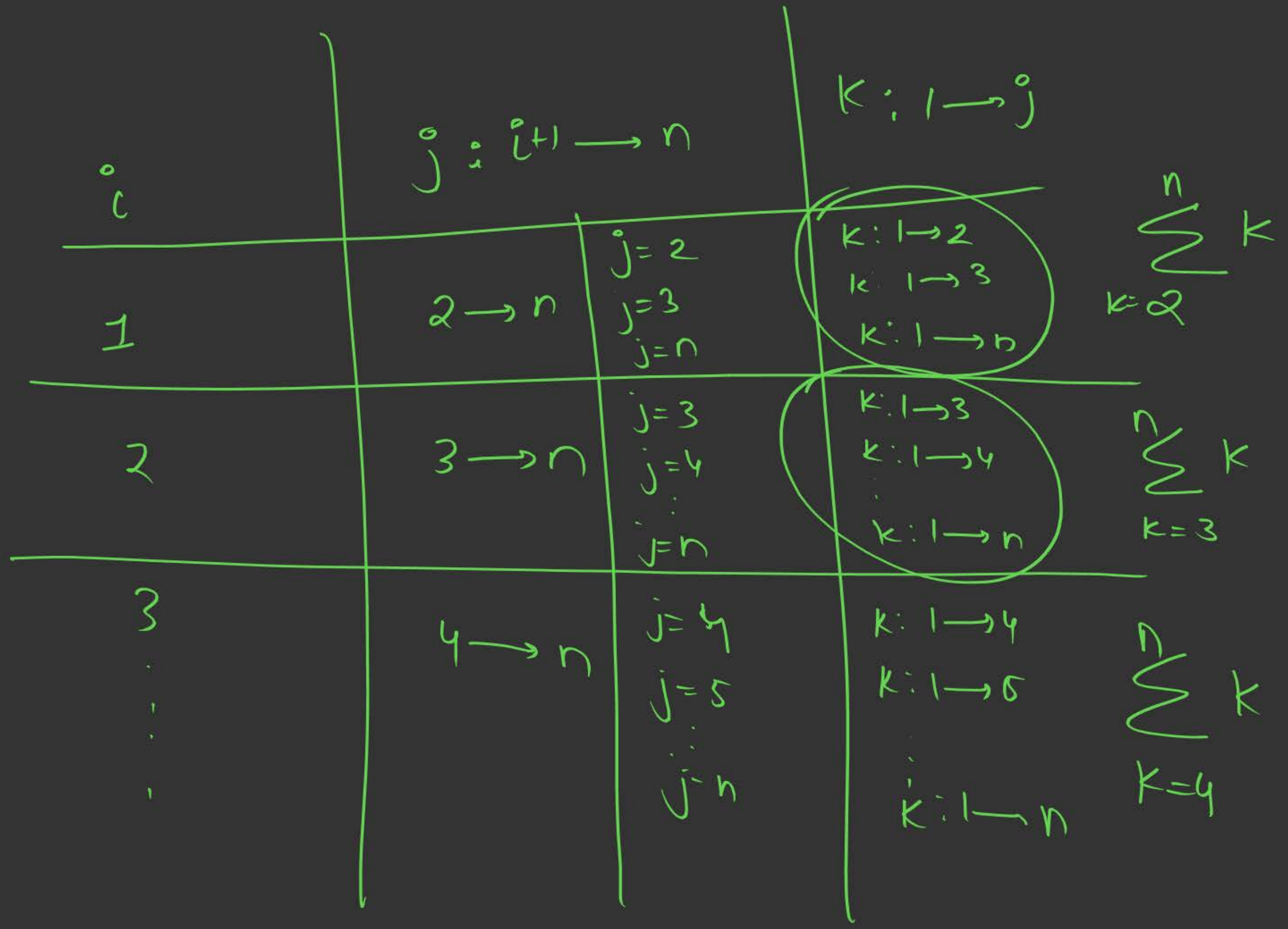
V.V. Adv

Bonus

#Q.
```
for i ← 1 to (n – 1)
    for j ← i + 1 to n
        for k ← 1 to j
            print ("*")
```

What is the number of times "*" gets printed

in terms of n


i
j : i+1 → n
k : 1 → j
1
2 → n
j = 2
j = 3
j = n
k : 1 → 2
k : 1 → 3
k : 1 → n
$\sum_{k=2}^{n} k$
2
3 → n
j = 3
j = 4
j = n
k : 1 → 3
k : 1 → 4
k : 1 → n
$\sum_{k=3}^{n} k$
3
4 → n
j = 4
j = 5
j = n
k : 1 → 4
k : 1 → 5
k : 1 → n
$\sum_{k=4}^{n} k$

Ans :

$$\left[\sum_{k=2}^{n} k + \sum_{k=3}^{n} k + \sum_{k=4}^{n} k \quad \cdots \quad \sum_{k=n}^{n} k \right]$$

Bonus



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Intro to DnC

Loop Adv

Non-DnC min-max



THANK - YOU